

Hand On 1

Nomor 1

Membuat array yang berkisar dari 0 hingga 2 dengan langkah sebesar 0.0001

pertama, mengimport library `numpy` untuk membuat array `t`. kemudian mengimport `matplotlib` untuk memvisualisasikan grafik dari data yang diproses. Kedua, membuat array `t` yang menggunakan `np.arange`. Kenapa tidak menggunakan `np.linspace`? karena nilai yang ingin kita capai yaitu 0 hingga 2 dengan selisih 0.0001.

- penggunaan `np.linspace` : (start, stop, num) : `num` ini adalah banyaknya jumlah yang diinginkan dalam interval start hingga stop
- penggunaan `np.arange` : (start, stop, step) : `step` ini adalah selisih antar elemen

dibawah ini adalah hasil dari array yang dibuat

```
In [117... import numpy as np
import matplotlib.pyplot as plt

t = np.arange(0, 2, 0.0001)

#menampilkan t indeks ke-3
print(f"t = {t}")
```

```
t = [0.0000e+00 1.0000e-04 2.0000e-04 ... 1.9997e+00 1.9998e+00 1.9999e+00]
```

Membuat sinyal - sinyal

1. $y_1 = 2 \cdot \sin(2\pi \cdot 3 \cdot t + 0)$: Ini adalah persamaan gelombang sinus dengan amplitudo 2, frekuensi 3 Hz, dan fase awal 0 radian.
2. $y_2 = 1 \cdot \cos(2\pi \cdot 4 \cdot t + \pi/4)$: Persamaan ini adalah gelombang kosinus dengan amplitudo 1, frekuensi 4 Hz, dan fase awal ($\pi/4$) radian.
3. $y_3 = -1 \cdot \sin(2\pi \cdot 5 \cdot t + \pi/2)$: Ini adalah persamaan gelombang sinus dengan amplitudo -1 (membalikkan gelombang), frekuensi 5 Hz, dan fase awal ($\pi/2$) radian.
4. $y_4 = 0.5 \cdot \cos(2\pi \cdot 6 \cdot t + \pi)$: Persamaan ini adalah gelombang kosinus dengan amplitudo 0.5, frekuensi 6 Hz, dan fase awal (π) radian.

dibawah adalah hasil dari sinyal - sinyal yang dibuat

```
In [118... # Mmembuat sinyal y1, y2, y3, dan y4
y1 = 2 * np.sin(2 * np.pi * 3 * t + 0)
y2 = 1 * np.cos(2 * np.pi * 4 * t + np.pi / 4)
y3 = -1 * np.sin(2 * np.pi * 5 * t + np.pi / 2)
y4 = 0.5 * np.cos(2 * np.pi * 6 * t + np.pi)

#menampilkan sinyal sinyal yang telah dibuat
print(f"y1 = {y1}")
print(f"y2 = {y2}")
```

```
print(f"y3 = {y3}")
print(f"y4 = {y4}")
```

```
y1 = [ 0.          0.00376991  0.0075398  ... -0.01130967 -0.0075398
      -0.00376991]
y2 = [0.70710678  0.7053274  0.70354356 ... 0.71241809 0.71065214 0.7088817 ]
y3 = [-1.          -0.99999507 -0.99998026 ... -0.99995559 -0.99998026
      -0.99999507]
y4 = [-0.5         -0.49999645 -0.49998579 ... -0.49996802 -0.49998579
      -0.49999645]
```

Nomor 2

Membuat perbandingan subplot

- Buat gambar dengan 4 subplot (grid 2x2) untuk membandingkan semua sinyal secara berdampingan.
- Setiap subplot harus berisi salah satu sinyal (y_1), (y_2), (y_3), dan (y_4).

Penjelasan: Membuat gambar yang berisi 4 grafik subplot (y_1 , y_2 , y_3 , y_4) dengan 2 baris dan 2 kolom

```
In [119... #pertama membuat tempat untuk plot dengan bentuk 2 baris dan 2 kolom
ax, fig = plt.subplots(2, 2, figsize=(20, 10))

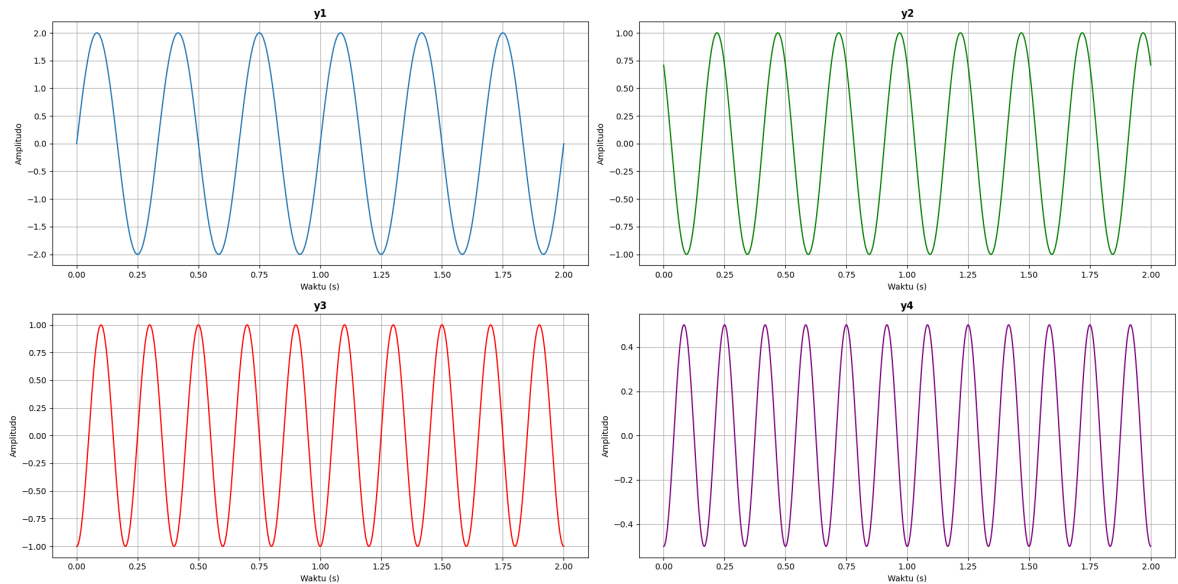
# Plot y1
fig[0, 0].plot(t, y1)
fig[0, 0].set_title(r'$\mathbf{y1}$')
fig[0, 0].grid(True)
fig[0, 0].set_xlabel('Waktu (s)')
fig[0, 0].set_ylabel('Amplitudo')

# Plot y2
fig[0, 1].plot(t, y2, color='green')
fig[0, 1].set_title(r'$\mathbf{y2}$')
fig[0, 1].grid(True)
fig[0, 1].set_xlabel('Waktu (s)')
fig[0, 1].set_ylabel('Amplitudo')

# Plot y3
fig[1, 0].plot(t, y3, color='red')
fig[1, 0].set_title(r'$\mathbf{y3}$')
fig[1, 0].grid(True)
fig[1, 0].set_xlabel('Waktu (s)')
fig[1, 0].set_ylabel('Amplitudo')

# Plot y4
fig[1, 1].plot(t, y4, color='purple')
fig[1, 1].set_title(r'$\mathbf{y4}$')
fig[1, 1].grid(True)
fig[1, 1].set_xlabel('Waktu (s)')
fig[1, 1].set_ylabel('Amplitudo')

# Adjust layout
plt.tight_layout()
plt.show()
```



In [120...

```
#contoh lain perubahan fase pergeseran
y6 = 2 * np.sin(2 * np.pi * 3 * t + np.pi/8) #22,5 derajat
y7 = 1 * np.cos(2 * np.pi * 4 * t + np.pi / 10) # 18 derajat
y8 = -1 * np.sin(2 * np.pi * 5 * t + np.pi / 12) # 15 derajat
y9 = 0.5 * np.cos(2 * np.pi * 6 * t + np.pi/ 18) # 10 derajat

#pertama membuat tempat untuk plot dengan bentuk 2 baris dan 2 kolom
ax, fig = plt.subplots(2, 2, figsize=(20, 10))

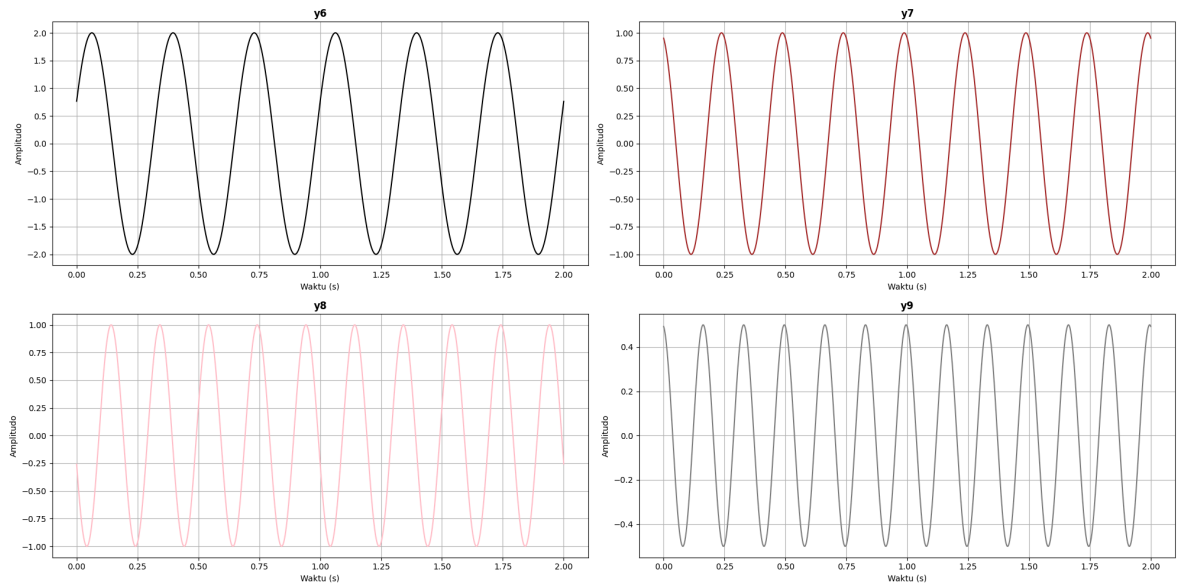
# Menampilkan sinyal y1
fig[0, 0].plot(t, y6, color= 'black')
fig[0, 0].set_title(r'$\mathbf{y6}$')
fig[0, 0].grid(True)
fig[0, 0].set_xlabel('Waktu (s)')
fig[0, 0].set_ylabel('Amplitudo')

# Menampilkan sinyal y2
fig[0, 1].plot(t, y7, color='brown')
fig[0, 1].set_title(r'$\mathbf{y7}$')
fig[0, 1].grid(True)
fig[0, 1].set_xlabel('Waktu (s)')
fig[0, 1].set_ylabel('Amplitudo')

# Menampilkan sinyal y3
fig[1, 0].plot(t, y8, color='pink')
fig[1, 0].set_title(r'$\mathbf{y8}$')
fig[1, 0].grid(True)
fig[1, 0].set_xlabel('Waktu (s)')
fig[1, 0].set_ylabel('Amplitudo')

# Menampilkan sinyal y4
fig[1, 1].plot(t, y9, color='grey')
fig[1, 1].set_title(r'$\mathbf{y9}$')
fig[1, 1].grid(True)
fig[1, 1].set_xlabel('Waktu (s)')
fig[1, 1].set_ylabel('Amplitudo')

# Adjust Layout
plt.tight_layout()
plt.show()
```



Nomor 3

Pertanyaan Analisis:

1. Berapa amplitudo dan frekuensi masing-masing sinyal?

Jawab:

- untuk sinyal y1, amplitudo = 2 dan frekuensi = 3
- untuk sinyal y2, amplitudo = 1 dan frekuensi = 4
- untuk sinyal y3, amplitudo = -1 dan frekuensi = 5
- untuk sinyal y4, amplitudo = 0.5 dan frekuensi = 6

-
2. Bagaimana pergeseran fase mempengaruhi posisi gelombang?

Jawab: jika dilihat pada gelombang-gelombang di atas, dapat disimpulkan bahwa fase mempengaruhi letak titik mulai gelombang, sehingga bentuk dan posisi gelombang juga berubah. Contohnya gambar gelombang y1 dengan y6. y1 yang memiliki fase 0 akan membuat gelombang yang mulai dari titik (0,0), sedang y6 memiliki fase ($\pi/8$) akan bergeser 22,5 derajat dari titik awal sehingga gelombang akan mulai dari sekitar (0, 0.75). Pergeseran fase tidak mengubah amplitudo atau frekuensi gelombang, tetapi mengubah titik awal dari siklus gelombang.

3. Bandingkan sinyal-sinyal dengan amplitudo yang berbeda dan diskusikan bagaimana amplitudo mempengaruhi tampilan gelombang.

Jawab: setiap sinyal yang dibuat memiliki amplitudo yang berbeda, jika dianalisis pada gambar gelombang di atas, tidak terlihat perbedaannya secara kasat mata, tetapi jika diteliti terdapat perbedaan, yaitu amplitudo sangat berpengaruh pada titik tertinggi dan titik terendah dari gelombang yang dihasilkan. Jika dilihat y_1 memiliki titik tertinggi 2 dan titik terendah -2, begitupun seterusnya bentuk sinyal yang dibuat akan memiliki tinggi rendah yang sama dengan amplitudo.

4. Bandingkan sinyal-sinyal dengan pergeseran fase yang berbeda dan diskusikan bagaimana pergeseran fase mempengaruhi tampilan gelombang.

Jawab:

Sinyal 1: $y_1 = 2 \cdot \sin(2\pi \cdot 3 \cdot t + 0)$

- Ini adalah gelombang sinusoidal dengan frekuensi 3 Hz (3 siklus per detik) dan memiliki Amplitudo = 2
- **Fase** = 0 radian (atau 0 derajat), artinya sinyal ini dimulai dari titik nol dan naik terlebih dahulu. Bentuk dasar ini dapat dianggap sebagai referensi untuk sinyal lain yang memiliki pergeseran fase.

Sinyal 2: $y_2 = 1 \cdot \cos(2\pi \cdot 4 \cdot t + \pi/4)$

- Gelombang cosinus dengan frekuensi 4 Hz dan memiliki Amplitudo = 1
- **Fase** = $(\pi/4)$ radian (45 derajat). Pergeseran fase ini membuat sinyal bergeser ke kiri pada sumbu waktu, menggeser puncak sinyal dari titik asalnya. Pergeseran fase ini mempengaruhi titik awal gelombang, sehingga puncaknya muncul lebih awal dibandingkan gelombang tanpa pergeseran fase. Ini berarti waktu untuk mencapai puncak pertama dipercepat dibandingkan dengan sinyal tanpa fase.

Sinyal 3: $y_3 = -1 \cdot \sin(2\pi \cdot 5 \cdot t + \pi/2)$

- Gelombang sinusoidal dengan frekuensi 5 Hz dan memiliki Amplitudo = 1 (nilai negatif mengindikasikan sinyal dibalik atau di-refleksi).
- **Fase** = Pergeseran fase 90° ($(\pi/2)$) dalam sinyal sinus berarti sinyal dimulai dari puncak positif (jika amplitudo positif) atau puncak negatif (jika amplitudo negatif seperti dalam sinyal ini). Karena amplitudo negatif, sinyal ini dimulai dari puncak negatif (dalam posisi rendah) dan bukannya dari titik nol seperti pada sinyal pertama. Ini menggeser seluruh sinyal, dan tampilan visualnya berbeda drastis dibandingkan sinyal tanpa fase.

Sinyal 4: $y_4 = 0.5 \cdot \cos(2\pi \cdot 6 \cdot t + \pi)$

- Gelombang cosinus dengan frekuensi 6 Hz dan memiliki Amplitudo = 0.5

- **Fase** = Pergeseran fase 180° (π) radian) menyebabkan gelombang dimulai pada puncak negatif dari sinyal cosinus. Sinyal ini adalah versi terbalik dari gelombang cosinus standar karena fase (π) radian menempatkan puncak negatif di tempat puncak positif seharusnya berada. Secara visual, sinyal ini akan tampak seperti kebalikan dari sinyal tanpa pergeseran fase pada posisi awalnya.

Kesimpulan:

- Pergeseran fase mengubah posisi relatif dari gelombang pada sumbu waktu, menggeser titik awal puncak atau lembah.
- Pergeseran kecil (misalnya 45°) membuat puncak gelombang terjadi lebih cepat atau lebih lambat.
- Pergeseran yang lebih besar, seperti 90° atau 180° , dapat membalik atau memutar gelombang secara signifikan, membuatnya tampak seperti sinyal baru yang dimulai dari puncak atau lembah yang berbeda.

Nomor 4

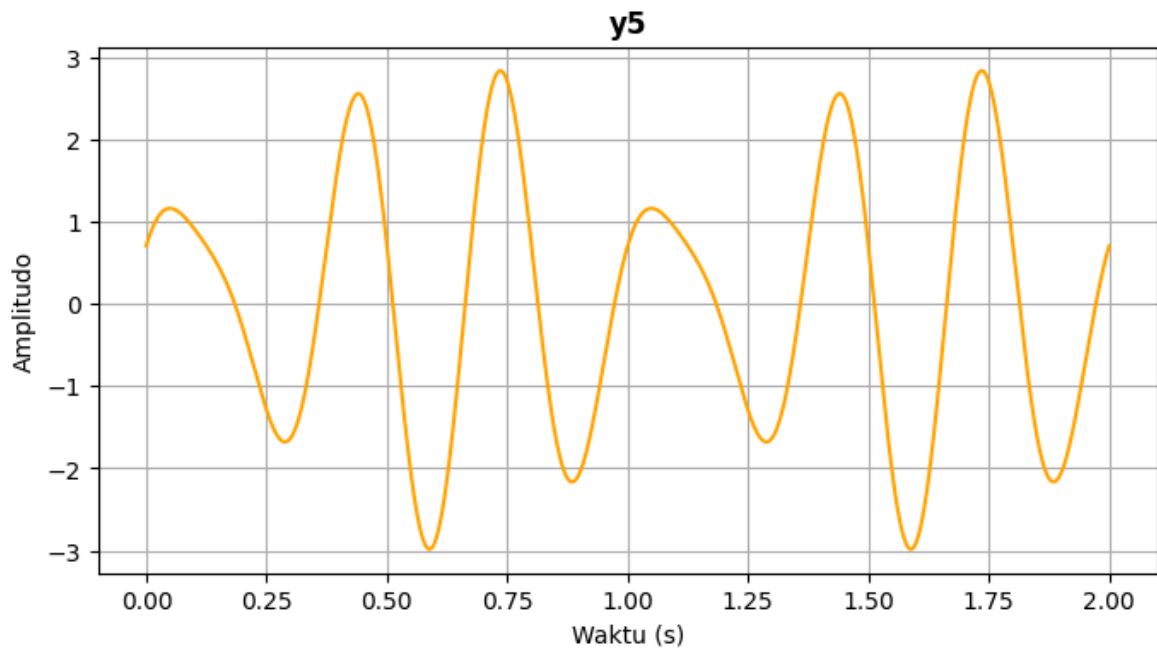
- Buat sinyal baru y_5 yang merupakan kombinasi dari y_1 dan y_2 yaitu, $y_5 = y_1 + y_2$.

Dibawah ini kita akan membuat sinyal gabungan. Pertama kita akan menjumlahkan sinyal y_1 dengan sinyal y_2 . kemudian kita menampilkan gelombang hasil dari penjumlahan y_1 dan y_2 .

In [121...

```
# Menggabungkan sinyal y1 dan y2
y5 = y1 + y2

# Menampilkan sinyal y5
plt.figure(figsize=(8, 4))
plt.plot(t, y5, color='orange')
plt.title(r'$\mathbf{y_5}$')
plt.xlabel('Waktu (s)')
plt.ylabel('Amplitudo')
plt.grid(True)
```



Plot y_5 dan diskusikan bagaimana kombinasi dua gelombang sinus/cosinus mempengaruhi bentuk gelombang yang dihasilkan

Jawab :

Definisi Gelombang:

- y_1 : Gelombang sinus dengan parameter yang telah ditentukan.
- y_2 : Gelombang cosinus dengan parameter yang telah ditentukan.
- y_5 : Kombinasi dari y_1 dan y_2 dengan menjumlahkan kedua gelombang tersebut.

Plot Gelombang: Pada grafik y_5 , kedua gelombang mempengaruhi bentuk gelombang yang dihasilkan.

Pengaruh Kombinasi:

- Interferensi Konstruktif: Ketika puncak dari kedua gelombang bertemu, mereka akan saling memperkuat, menghasilkan amplitudo yang lebih besar.
- Interferensi Destruktif: Ketika puncak dari satu gelombang bertemu dengan lembah dari gelombang lain, mereka akan saling melemahkan, menghasilkan amplitudo yang lebih kecil atau bahkan nol.

Kesimpulan: Dengan mengkombinasi sinyal y_1 dan y_2 , kita dapat melihat bagaimana bentuk gelombang yang dihasilkan dipengaruhi oleh superposisi dari dua gelombang sinus dan cosinus.

Nomor 5

5. Buktikanlah bahwa proses downsampling (resampling dengan laju sampling yang lebih rendah) dapat menghilangkan informasi dari sinyal asli. Untuk melakukan hal ini, gunakan sinyal ECG sintesis (dengan method `nk.ecg_simulate`) sesuai spesifikasi berikut:

- Durasi: 080
- Sampling Rate: 150 Hz
- Noise Level: 0.80
- Heart Rate: 80 BPM
- Random State: 03717

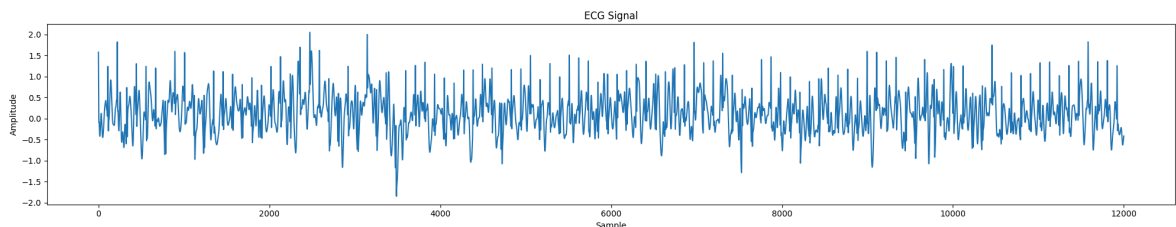
Lakukan downsampling dari 150Hz ke 100Hz, 50Hz, 25Hz, 10Hz, hingga 5Hz.

In [122...

```
# mengimport library yang dibutuhkan
import matplotlib.pyplot as plt
import numpy as np
import neurokit2 as nk
import pywt
from scipy import signal
from obspy.core import UTCDateTime
from obspy.clients.syngine import Client

# membuat sinyal ECG yang memiliki durasi 80 detik, frekuensi sampling 150 Hz, h
ecg_signal = nk.ecg_simulate(duration=80, sampling_rate=150, heart_rate=80, nois

# menampilkan sinyal ECG
plt.figure(figsize=(20, 4))
plt.plot(ecg_signal)
plt.title("ECG Signal")
plt.ylabel("Amplitude")
plt.xlabel("Sample")
plt.tight_layout()
plt.show()
```

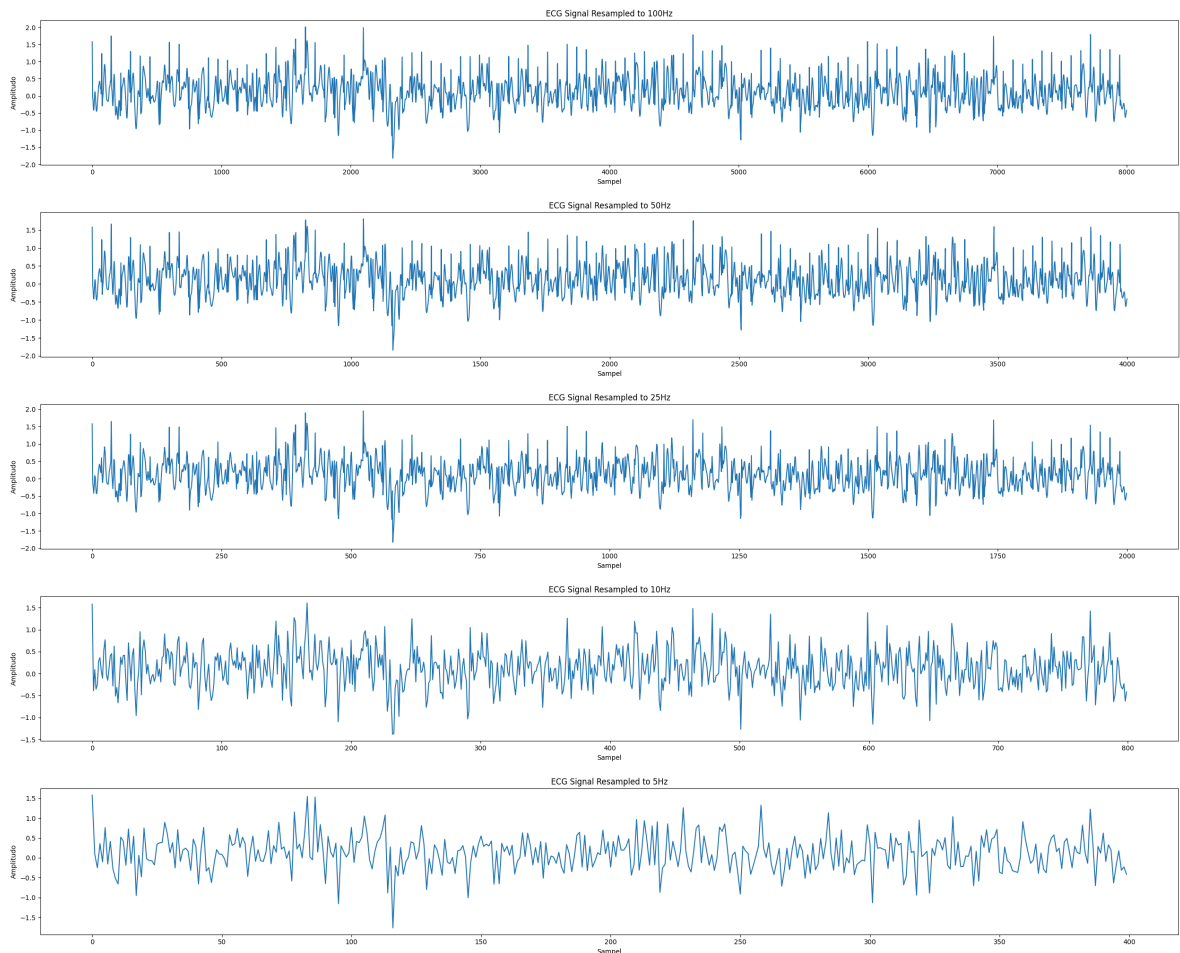


In [123...

```
# Melakukan downsampling sinyal ECG dari 150Hz menjadi 100Hz, 50Hz, 25Hz, 10Hz,
fs_target = [100, 50, 25, 10, 5] #deklarasi frekuensi sampling
durasi_asli = 80 # dalam detik

# membuat perulangan untuk melakukan resampling
for rate in fs_target:
    panjang_elemen_setelah_resampling = rate * durasi_asli
    time_axis = np.linspace(0, len(ecg_signal), panjang_elemen_setelah_resamplin
    downsampling = np.interp(time_axis, np.arange(len(ecg_signal)), ecg_signal)

    plt.figure(figsize=(25, 4))
    plt.plot(downsampling)
    plt.title(f"ECG Signal Resampled to {rate}Hz")
    plt.ylabel("Amplitudo")
    plt.xlabel("Sampel")
    plt.tight_layout()
    plt.show()
```

Jelaskan apa yang terjadi dan buktikan bahwa semakin rendah sampling frequency (f_s) maka sinyal akan semakin terdistorsi dan terdapat **Aliasing** pada sinyal hasil downsampling. Jelaskan apa itu **Aliasing** Jawab : Aliasing adalah fenomena yang terjadi ketika sinyal yang di-sampling tidak memenuhi kriteria, yaitu ketika sampling frequency (f_s) kurang dari dua kali frekuensi tertinggi dari sinyal asli. Akibatnya, sinyal yang dihasilkan setelah downsampling akan terdistorsi dan tidak merepresentasikan sinyal asli dengan benar, sehingga menciptakan artefak yang tidak ada dalam sinyal asli. Ini menyebabkan sinyal yang dihasilkan setelah downsampling menjadi tidak akurat.

pada soal nomor 5 diatas, kita melakukan memiliki sinyal ECG yang memiliki durasi 80 detik, frekuensi sampling 150 Hz, heart rate 80 bpm, noise 0.80, dan random state 3717. Lalu kita melakukan downsampling dari 150Hz menjadi 100Hz, 50Hz, 25Hz, 10Hz dan 5Hz. Jika kita analisis sinyal resample, tidak begitu terlihat jelas perbedaan yang dihasilkan. namun, jika dilihat lebih jauh, **semakin kecil frekuensi sampling, maka semakin sedikit sampel yang dihasilkan, maka akan semakin jauh antar titik sampel yang dihasilkan sehingga gelombang yang dihasilkan tidak semulus jika frekuensinya lebih tinggi. begitupun sebaliknya, jika frekuensi makin tinggi maka akan semakin banyak sampel yang dihasilkan, maka akan semakin rapat antar titik sampel sehingga gelombang akan semakin mulus.** Untuk pembuktian lebih jelas, **dibawah ini contoh lain downsampling yg dibuat github copilot:**

Analisis Hasil dari contoh:

$f_s = 50$ Hz: Sampling frequency ini cukup tinggi untuk menangkap sinyal asli dengan baik, sehingga sinyal hasil downsampling masih cukup akurat. $f_s = 20$ Hz: Mulai terlihat

sedikit distorsi, tetapi sinyal masih dapat dikenali. $f_s = 10$ Hz: Terjadi aliasing yang signifikan, sinyal hasil downsampling sangat terdistorsi dan tidak lagi merepresentasikan sinyal asli.

Kesimpulan:

Semakin rendah sampling frequency (f_s), semakin besar kemungkinan terjadinya aliasing, yang menyebabkan sinyal hasil downsampling menjadi terdistorsi.

In [124...

```
# Parameter sinyal asli
f = 5 # Frekuensi sinyal asli (Hz)
t = np.linspace(0, 1, 1000, endpoint=False) # Waktu
x = np.sin(2 * np.pi * f * t) # Sinyal sinusoidal

plt.figure(figsize=(10, 4))
plt.plot(t, x)
plt.title('Sinyal Asli')
plt.xlabel('Waktu (s)')
plt.ylabel('Amplitudo')
plt.show()

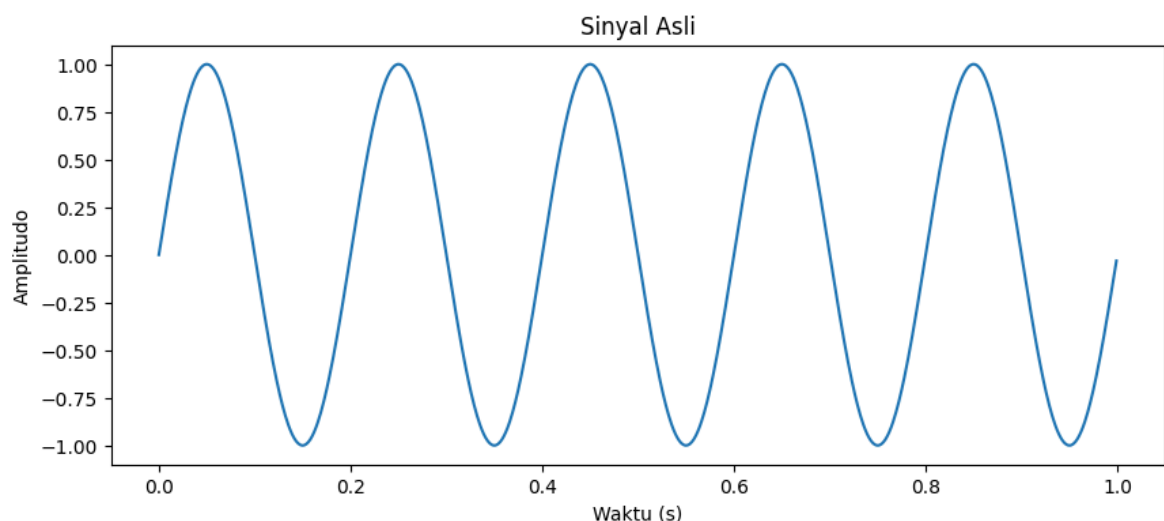
def downsample_signal(x, factor):
    return x[::factor]

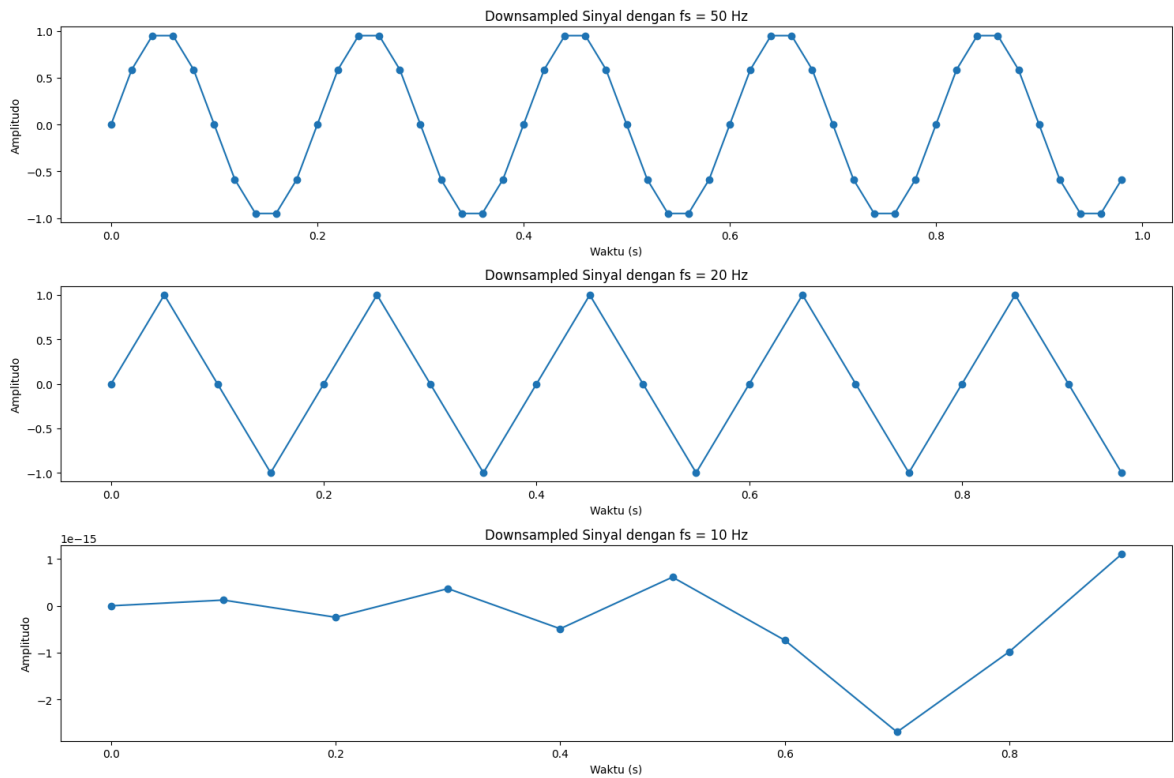
# Sampling frequency yang berbeda
fs_list = [50, 20, 10] # Hz

plt.figure(figsize=(15, 10))
for i, fs in enumerate(fs_list, 1):
    factor = 1000 // fs
    x_downsampled = downsample_signal(x, factor)
    t_downsampled = t[::factor]

    plt.subplot(len(fs_list), 1, i)
    plt.plot(t_downsampled, x_downsampled, 'o-')
    plt.title(f'Downsampled Sinyal dengan fs = {fs} Hz')
    plt.xlabel('Waktu (s)')
    plt.ylabel('Amplitudo')

plt.tight_layout()
plt.show()
```





Soal Nomor 6

Pengertian Order pada Filtering

Order dalam konteks filtering merujuk pada jumlah elemen atau koefisien dalam filter. Dalam filter digital, order menentukan kompleksitas dan karakteristik frekuensi dari filter tersebut. Semakin tinggi order maka semakin tajam dan lebih kompleks filter tersebut dalam memisahkan atau menghaluskan sinyal.

Pengaruh Mengubah Nilai Order

Mengubah nilai order akan mempengaruhi respons frekuensi dari filter. Filter dengan order yang lebih tinggi akan memiliki transisi yang lebih tajam antara frekuensi yang diinginkan dan yang tidak diinginkan, tetapi juga akan lebih kompleks dan mungkin memerlukan lebih banyak komputasi. Sebaliknya, filter dengan order yang lebih rendah akan lebih sederhana dan lebih cepat, tetapi mungkin tidak seefektif dalam memisahkan frekuensi.

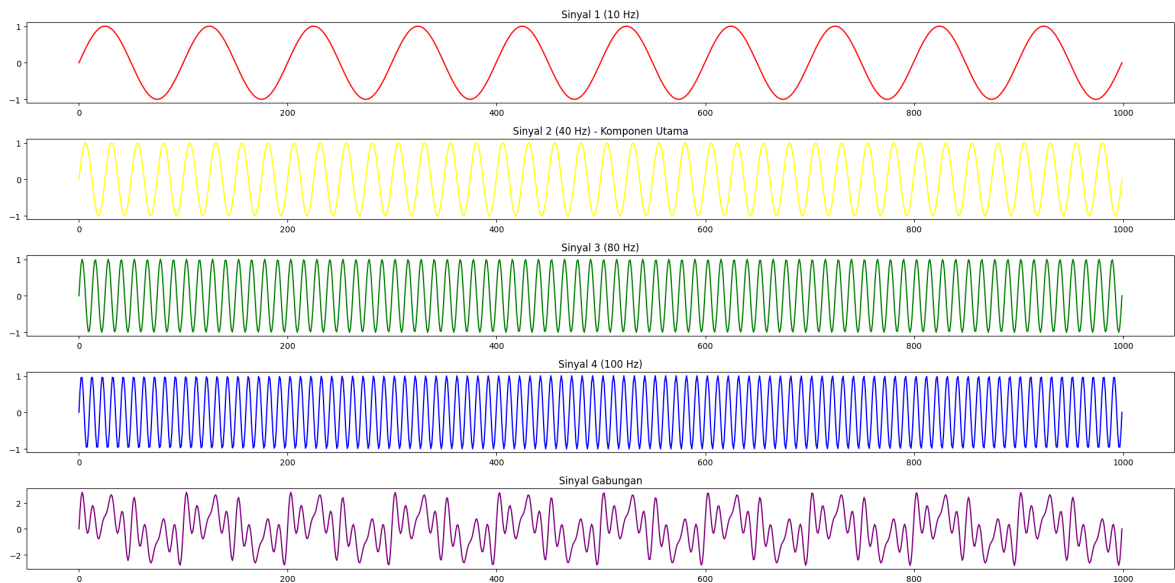
In [125...

```
# Membuat sinyal dengan frekuensi 10, 40, 80, dan 100 Hz
fs = 1000 # Frekuensi sampling
time_axis = np.linspace(0, 1, fs) # Waktu

sinyal_1 = np.sin(2 * np.pi * 10 * time_axis) # 10 Hz
sinyal_2 = np.sin(2 * np.pi * 40 * time_axis) # 40 Hz
sinyal_3 = np.sin(2 * np.pi * 80 * time_axis) # 80 Hz
sinyal_4 = np.sin(2 * np.pi * 100 * time_axis) # 100 Hz
sinyal_gabungan = sinyal_1 + sinyal_2 + sinyal_3 + sinyal_4 # Sinyal gabungan

# Menampilkan sinyal
fig, ax = plt.subplots(5, 1, figsize=(20, 10))
```

```
ax[0].plot(sinyal_1, color='red')
ax[0].set_title("Sinyal 1 (10 Hz)")
ax[1].plot(sinyal_2, color='yellow')
ax[1].set_title("Sinyal 2 (40 Hz) - Komponen Utama")
ax[2].plot(sinyal_3, color='green')
ax[2].set_title("Sinyal 3 (80 Hz)")
ax[3].plot(sinyal_4, color='blue')
ax[3].set_title("Sinyal 4 (100 Hz)")
ax[4].plot(sinyal_gabungan, color='purple')
ax[4].set_title("Sinyal Gabungan")
plt.tight_layout()
plt.show()
```



In [126...

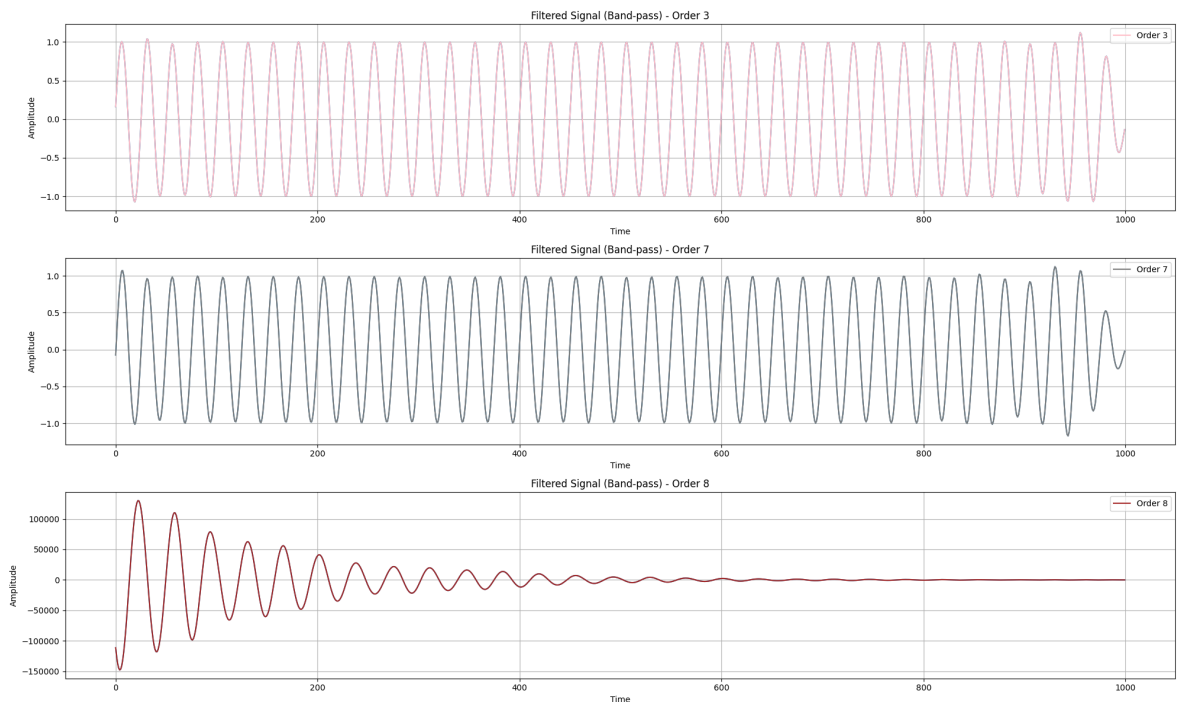
```
# Membuat Band-pass filter
cutoff_low = 27 # Frekuensi cutoff dalam Hz
cutoff_high = 50 # Frekuensi cutoff dalam Hz
orders = [3, 7, 8] # Orde filter yang akan dibandingkan
colors = ['pink', 'grey', 'brown']

# Menampilkan sinyal yang telah difilter
plt.figure(figsize=(20, 12))

for i, order in enumerate(orders, 1):
    b, a = signal.butter(order, [cutoff_low, cutoff_high], fs=fs, btype='band',
        signal_filt_band = signal.filtfilt(b, a, sinyal_gabungan)

    plt.subplot(len(orders), 1, i)
    plt.plot(signal_filt_band)
    plt.plot(signal_filt_band, label=f'Order {order}', color=colors[i-1])
    plt.title(f"Filtered Signal (Band-pass) - Order {order}")
    plt.ylabel("Amplitude")
    plt.xlabel("Time")
    plt.grid(True)
    plt.legend()

plt.tight_layout()
plt.show()
```



Analisis Hasil Visualisasi Plot dari Ketiga Grafik di Atas

1. Sinyal Gabungan (Sinyal 1, Sinyal 2, Sinyal 3, Sinyal 4):

- **Sinyal 1 (10 Hz):** Sinyal ini memiliki frekuensi rendah dan terlihat sebagai gelombang sinusoidal yang halus. Amplitudo sinyal ini relatif kecil dibandingkan dengan sinyal lainnya.
- **Sinyal 2 (40 Hz):** Sinyal ini memiliki frekuensi yang lebih tinggi dibandingkan dengan Sinyal 1 dan terlihat lebih rapat. Amplitudo sinyal ini lebih besar dan menjadi komponen utama dalam sinyal gabungan.
- **Sinyal 3 (80 Hz):** Sinyal ini memiliki frekuensi yang lebih tinggi lagi dan terlihat sangat rapat. Amplitudo sinyal ini lebih kecil dibandingkan dengan Sinyal 2.
- **Sinyal 4 (100 Hz):** Sinyal ini memiliki frekuensi tertinggi di antara keempat sinyal. Amplitudo sinyal ini juga relatif kecil.
- **Sinyal Gabungan:** Kombinasi dari keempat sinyal ini menghasilkan sinyal yang kompleks dengan berbagai frekuensi dan amplitudo. Sinyal ini menunjukkan bagaimana berbagai komponen frekuensi dapat digabungkan untuk membentuk sinyal yang lebih kompleks.

2. Filtered Signal (Band-pass) - Order 3, 7, 8:

- **Order 3:** Filter dengan order rendah ini menghasilkan sinyal yang masih memiliki beberapa komponen frekuensi yang tidak diinginkan. Transisi antara frekuensi yang diinginkan dan yang tidak diinginkan tidak terlalu tajam.
- **Order 7:** Filter dengan order menengah ini menghasilkan sinyal yang lebih bersih dibandingkan dengan order 3. Transisi antara frekuensi yang diinginkan dan yang tidak diinginkan lebih tajam.
- **Order 8:** Filter dengan order tinggi ini menghasilkan sinyal yang paling bersih dengan transisi yang sangat tajam antara frekuensi yang diinginkan dan yang tidak diinginkan. Namun, filter dengan order tinggi juga dapat menampilkan distorsi atau artefak lain dalam sinyal.

Kesimpulan

- **Frekuensi dan Amplitudo:** Frekuensi yang lebih tinggi menghasilkan sinyal yang lebih rapat, sementara amplitudo menentukan tinggi rendahnya gelombang.
- **Filter Order:** Order filter yang lebih tinggi menghasilkan transisi yang lebih tajam dan sinyal yang lebih bersih, tetapi juga dapat memperkenalkan distorsi. Pemilihan order filter harus mempertimbangkan trade-off antara kompleksitas dan efektivitas dalam memisahkan frekuensi yang diinginkan.

Nomor 7

Eksperimen Filter Band-Pass pada Sinyal Respirasi

Latar Belakang

Sinyal respirasi (pernapasan) adalah sinyal biologis yang mencerminkan aktivitas pernapasan seseorang. Sinyal ini biasanya memiliki frekuensi yang lebih rendah dibandingkan dengan sinyal biologis lainnya seperti sinyal jantung (ECG). Untuk memproses sinyal respirasi, kita sering kali perlu menghilangkan noise dan komponen frekuensi yang tidak diinginkan. Salah satu cara untuk melakukan ini adalah dengan menggunakan filter band-pass.

Frekuensi pernapasan manusia berkisar:

- **Saat Normal :** berkisar antara 0.2 Hz hingga 0.33 Hz
- **Saat Ketakutan:** berkisar antara 0.42 Hz hingga 0.58 Hz
- **Saat Berlari :** berkisar antara 0.5 HZ hingga 0.67 Hz

In [127...

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt

# Parameter
fs = 100 # Frekuensi sampling (Hz)
duration = 10 # Durasi sinyal dalam detik
time_axis = np.linspace(0, duration, fs * duration)

freq_normal = 0.25 # fekuensi pernapasan normal
freq_ketakutan = 0.5 # frekuensi pernapasan saat ketakutan
freq_berlari = 0.8 # frekuensi pernapasan saat berlari

# Buat setiap sinyal
normal_signal = np.sin(2 * np.pi * freq_normal * time_axis)
ketakutan_signal = np.sin(2 * np.pi * freq_ketakutan * time_axis)
berlari_signal = np.sin(2 * np.pi * freq_berlari * time_axis)

# Gabungkan sinyal untuk membuat sinyal pernapasan gabungan
combined_signal = normal_signal + berlari_signal + ketakutan_signal

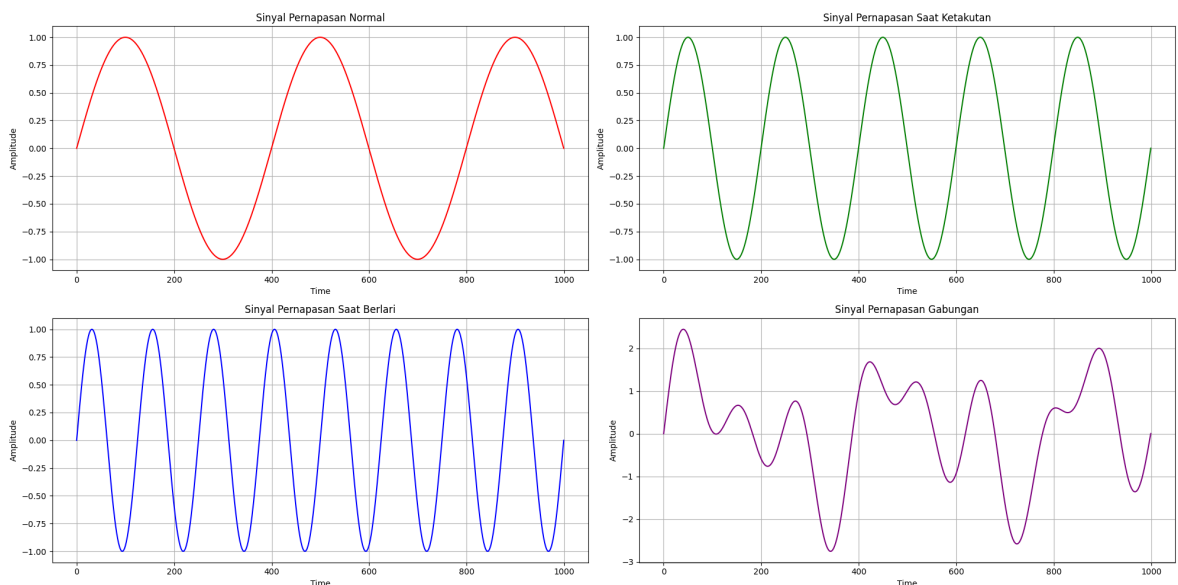
# Menampilkan sinyal
```

```

fig, ax = plt.subplots(2, 2, figsize=(20, 10))
ax[0, 0].plot(normal_signal, color='red')
ax[0, 0].set_title("Sinyal Pernapasan Normal")
ax[0, 0].set_ylabel("Amplitude")
ax[0, 0].set_xlabel("Time")
ax[0, 0].grid(True)
ax[0, 1].plot(ketakutan_signal, color='green')
ax[0, 1].set_title("Sinyal Pernapasan Saat Ketakutan")
ax[0, 1].set_ylabel("Amplitude")
ax[0, 1].set_xlabel("Time")
ax[0, 1].grid(True)
ax[1, 0].plot(berlari_signal, color='blue')
ax[1, 0].set_title("Sinyal Pernapasan Saat Berlari")
ax[1, 0].set_ylabel("Amplitude")
ax[1, 0].set_xlabel("Time")
ax[1, 0].grid(True)
ax[1, 1].plot(combined_signal, color='purple')
ax[1, 1].set_title("Sinyal Pernapasan Gabungan")
ax[1, 1].set_ylabel("Amplitude")
ax[1, 1].set_xlabel("Time")
ax[1, 1].grid(True)

plt.tight_layout()
plt.show()

```



Penentuan Frekuensi Cutoff

sinyal yang akan kita ambil adalah sinyal saat merasa ketakutan sehingga **Cutoff Low** di **0.42 HZ** dan **Cutoff High** di **0.58 HZ**

Kesimpulan

Dari hasil percobaan, kita dapat menyimpulkan bahwa penggunaan filter band-pass dengan cutoff low di **0.42 Hz** dan cutoff high di **0.58 Hz** berhasil mengekstraksi sinyal pernapasan saat ketakutan dari sinyal gabungan. Filter ini efektif dalam menghilangkan komponen frekuensi yang tidak diinginkan dan mempertahankan komponen frekuensi yang relevan dengan kondisi ketakutan. Hal ini menunjukkan bahwa filter band-pass dapat digunakan untuk memisahkan sinyal berdasarkan frekuensi

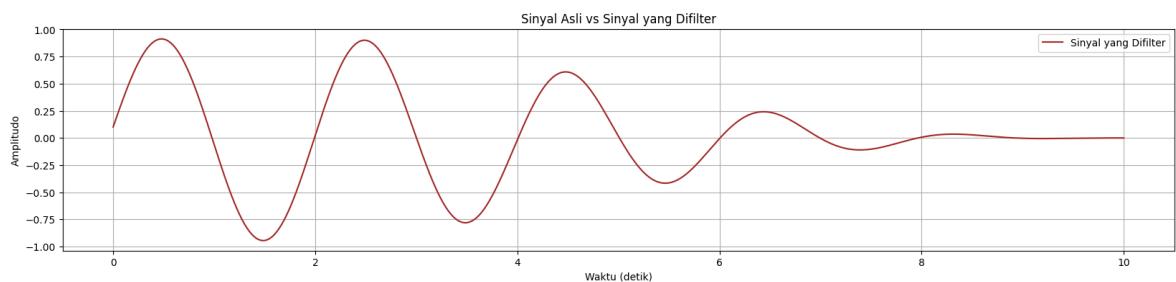
tertentu, sehingga memungkinkan analisis yang lebih mendalam terhadap kondisi yang berbeda.

In [128...

```
# Definisikan fungsi filter bandpass
def bandpass_filter(data, lowcut, highcut, fs, order=4):
    nyquist = 0.5 * fs
    low = lowcut / nyquist
    high = highcut / nyquist
    b, a = butter(order, [low, high], btype='band')
    y = filtfilt(b, a, data)
    return y

# Terapkan filter bandpass
filtered_signal = bandpass_filter(combined_signal, lowcut=0.42, highcut=0.58, fs

# Plot sinyal asli dan sinyal yang difilter
plt.figure(figsize=(20, 4))
plt.plot(time_axis, filtered_signal, label="Sinyal yang Difilter", color='brown')
plt.title("Sinyal Asli vs Sinyal yang Difilter")
plt.xlabel("Waktu (detik)")
plt.ylabel("Amplitudo")
plt.legend()
plt.grid()
plt.show()
```



Referensi

- Github co-pilot
- ChatGPT : <https://chatgpt.com/share/66fe5c02-bfd4-8001-b1b7-8aaab2e5266a>